

Faktoryzacja QR z rotacji Givensa

Rachunek Macierzowy — program 6

Kacper Bujak, Wojciech Maćkowiak

9 maja 2026

1 Cel

Zaimplementować rozkład QR macierzy kwadratowej przy pomocy rotacji Givensa i porównać wyniki z funkcją `qr` z Octave/MATLAB. Rozmiar macierzy wynika z polecenia: n jest sumą miesiąca i dnia urodzenia; w skrypcie `solution.m` ustawia się miesiąc oraz dzień (w raporcie aktualnie $n = 14$).

2 Wstęp teoretyczny

2.1 Rozkład QR

Dla macierzy prostokątnej $A \in \mathbb{R}^{m \times n}$ o pełnym rzędzie kolumnowym istnieje rozkład

$$A = QR, \quad (1)$$

gdzie $Q \in \mathbb{R}^{m \times m}$ jest ortogonalna ($Q^T Q = I$), a $R \in \mathbb{R}^{m \times n}$ jest górnotrójkątna (z pojęciem „górnotrójkątnej” dla prostokątnych macierzy — zera pod główną przekątną). W przypadku kwadratowym $m = n$ macierz R jest klasyczną górnotrójkątną.

Rozkład QR jest podstawą stabilnego rozwiązywania układów z nadokreśleniem (np. problem najmniejszych kwadratów), ortogonalizacji baz oraz wielu algorytmów numerycznych (wartości własne, SVD jako etapy pośrednie). Sam Q przechowuje informację o „obrocie układu współrzędnych”, a R — o kombinacji kolumn po tej zmianie bazy.

2.2 Niejednoznaczność znaku

Jeżeli $A = QR$, to dla pewnych macierzy diagonalnych D ze znakami ± 1 na przekątnej nadal $A = (QD)(D^{-1}R)$ i iloczyn pozostaje poprawnym rozkładem QR . W praktyce procedury biblioteczne mogą więc zwracać inne znaki kolumn Q lub wierszy R niż „nasza” procedura, choć rekonstrukcja QR powinna zgadzać się z A z dokładnością do błędu zaokrągleń.

2.3 Rotacje Givensa

Rotacja Givensa w $\mathbb{R}^m$ to macierz ortogonalna różniąca się od I tylko na czterech pozycjach (p, p) , (p, q) , (q, p) , (q, q) — odpowiada obrotowi w płaszczyźnie rozciągniętej przez wektory bazowe e_p, e_q . Zastosowanie takiej macierzy z lewej strony do A miesza ze sobą dwa wiersze macierzy.

Chcemy dobrać obrót tak, by wyzerować wybrany element w jednym z wierszy przy niezerowej „gałęzi” drugiego wiersza. Dla dwuwymiarowego wektora $(a, b)^T$ istnieją c, s ($c^2 + s^2 = 1$), że $\begin{bmatrix} c & s \\ -s & c \end{bmatrix} (a, b)^T = (r, 0)^T$. To jest dokładnie krok konstrukcji R w eliminacji podprzekątnej.

2.4 Budowa QR jako iloczyn rotacji

Jeśli kolejno od lewej mnożymy A przez rotacje zmiatające podprzekątną część kolejnych kolumn, otrzymujemy macierz górnotrójkątną:

$$G_k \cdots G_2 G_1 A = R.$$

Iloczyn $G_1^T G_2^T \cdots G_k^T$ (przy zapisie naszej implementacji odpowiada to składaniu czynników na macierz Q) jest ortogonalny jako iloczyn macierzy ortogonalnych. W ćwiczeniu interesuje nas wariant z zerowaniem od dołu w obrębie każdej kolumny — klasyczny schemat dla QR metodą Givensa na macierzu gęstej.

2.5 Złożoność (rzędu)

Dla gęstej macierzy $n \times n$ jest $O(n^2)$ potencjalnych rotacji (pętle po kolumnach i wierszach), a każda rotacja kosztuje $O(n)$ aktualizacji wierszy/kolumn — łącznie koszt rzędu $O(n^3)$ operacji zmiennoprzecinkowych (porównywalnie z eliminacją Gaussa lub QR Householderem). Przy macierzach rzadkich można pomijać wiele rotacji — wtedy Givens bywa korzystniejszy strukturalnie; tutaj skupiamy się na gęstym przypadku testowym.

3 Opis metody

3.1 Współczynniki obrotu

Niech $a = R(j, j)$ i $b = R(i, j)$ będą elementami pivotu oraz wiersza niżej w tej samej kolumnie ($i > j$). Przyjmujemy $r = \sqrt{a^2 + b^2}$, $c = a/r$, $s = b/r$ (w kodzie hypot (a, b)). Lewe mnożenie przez obrót w płaszczyźnie (j, i) aktualizuje dwa wiersze R oraz (przy naszej konwencji akumulacji) odpowiednio miesza dwie kolumny macierzy Q .

3.2 Eliminacja

Start: $R := A$, $Q := I$. Dla $j = 1, \dots, \min(m, n)$ i $i = m, m-1, \dots, j+1$ stosujemy powyższy obrót, dopóki część ściśle podprzekątna w kolumnie j nie zniknie (numerycznie: gdy $a = b = 0$, obrót pomijamy).

4 Pseudokod

```

Wejście: A (m x n)
R := A; Q := I_m
dla j = 1 : min(m, n)
  dla i = m, m-1, ..., j+1
    a := R(j, j); b := R(i, j)
    jeśli sqrt(a^2+b^2)=0: koniec obrotu dla tego i
    r := hypot(a, b); c := a/r; s := b/r

```

```

R(j,:) := c*R(j,:) + s*R(i,:)
R(i,:) := -s*R(j,:) + c*R(i,:)
Q(:,j) := c*Q(:,j) + s*Q(:,i)
Q(:,i) := -s*Q(:,j) + c*Q(:,i)

```

5 Pliki

Funkcja `qr_givens.m`, skrypt `solution.m` (parametry n , losowanie, normy). Wyniki liczbowe ładują w katalogu LaTeX/ jako makra oraz dwa pliki CSV z danymi pod wykresy; Python w `analysis/` zapisuje PDF-y do LaTeX/figures/.

6 Fragmenty kodu

```

1 function [Q, R] = qr_givens(A)
2 % QR przez obroty Givensa (lewa strona). A prostokątna m x n.
3 [m, n] = size(A);
4 R = A;
5 Q = eye(m);
6 for j = 1:min(m, n)
7     for i = m:-1:(j + 1)
8         a = R(j, j);
9         b = R(i, j);
10        if hypot(a, b) == 0
11            continue;
12        endif
13        r = hypot(a, b);
14        c = a / r;
15        s = b / r;
16        row_j = c * R(j, :) + s * R(i, :);
17        row_i = -s * R(j, :) + c * R(i, :);
18        R(j, :) = row_j;
19        R(i, :) = row_i;
20        col_j = c * Q(:, j) + s * Q(:, i);
21        col_i = -s * Q(:, j) + c * Q(:, i);
22        Q(:, j) = col_j;
23        Q(:, i) = col_i;
24    endfor
25 endfor
26 endfunction

```

Listing 1: solution.m (fragment)

```

1 [Qg, Rg] = qr_givens(A);
2 err_recon = norm(A - Qg * Rg, 'fro');
3 err_orth = norm(Qg' * Qg - eye(n), 'fro');
4 [Ql, Rl] = qr(A);
5 err_lib = norm(A - Ql * Rl, 'fro');

```

7 Wyniki

7.1 Przykład numeryczny dla $n = 14$

Macierz A losowa, ziarno generatora w `solution.m`. Dla takiego jednego przypadku mierzymy rekonstrukcję, ortogonalność Q oraz to, czy R jest faktycznie górnotrójkątna; dodatkowo ten sam błąd rekonstrukcji dla `qr(A)`.

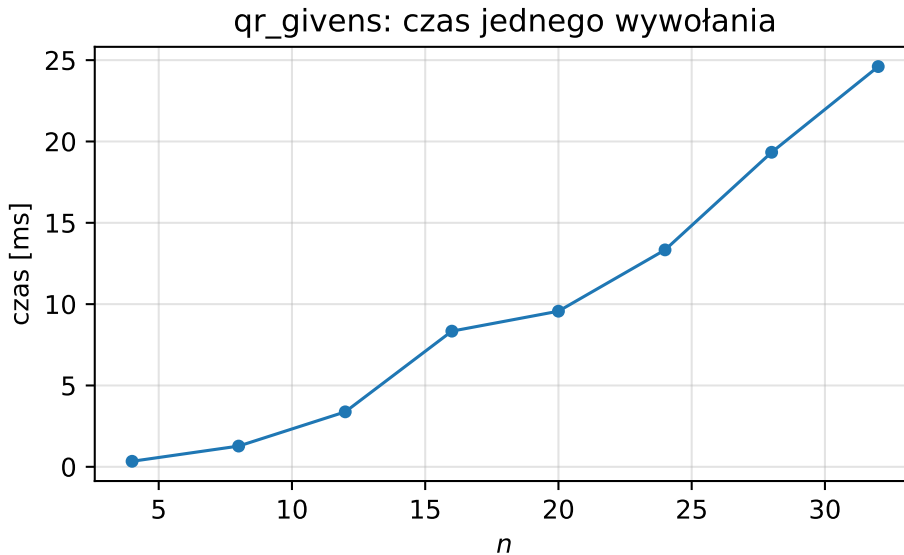
Tabela 1: Normy Frobeniusa wybranych residuów (jedna losowa macierz $n \times n$).

wielkość	wartość
$\ A - QR\ _F$ (Givens)	3.043049e-15
$\ Q^T Q - I\ _F$	1.989255e-15
$\ \text{tril}(R, -1)\ _F$	2.598154e-16
$\ A - QR\ _F$ (qr)	2.829375e-15

Wszystkie trzy pierwsze wiersze są rzędu 10^{-15} – 10^{-16} , czyli okolicy błędu maszynowego przy arytmetyce double. Czwarty wiersz pokazuje, że biblioteczny qr zachowuje się tak samo co do rekonstrukcji A (z dokładnością do tej samej skali numerycznej).

7.2 Czas obliczeń w funkcji n

Na rysunku 1 widać czas pojedynczego wywołania `qr_givens` dla losowej macierzy $n \times n$. Krzywa nie jest gładka (pojedynczy pomiar, narzut systemowy), ale trend rośnie wyraźnie szybciej niż liniowo — zgodnie z oczekiwaniem kosztu rzędu $O(n^3)$ dla gęstych macierzy.

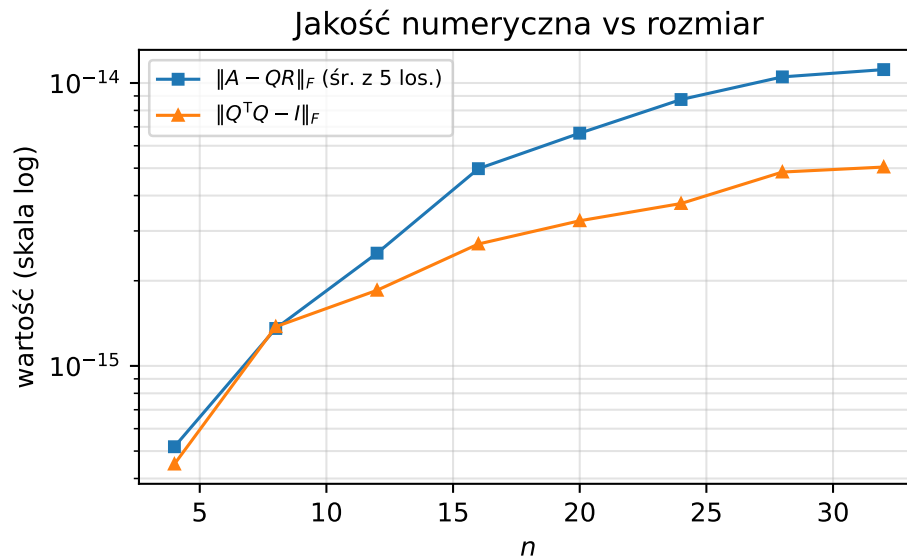


Rysunek 1: Czas jednego wywołania `qr_givens` vs. n .

7.3 Błędy numeryczne vs. n

Drugie doświadczenie (rysunek 2) pokazuje średnią z pięciu losowych macierzy dla każdego n : $\|A - QR\|_F$ oraz $\|Q^T Q - I\|_F$. Na osi pionowej jest skala logarytmiczna. Punkty ukła-

dają się w okolicy tej samej rzędu wielkości niezależnie od n w badanym zakresie — typowe zachowanie dla stabilnej procedury: błąd zaokrągleń nie „wybuchuje” gwałtownie przy rozsądnym n , tylko oscyluje w granicach precyzji.



Rysunek 2: Średnie błędy (5 losowań na każde n): rekonstrukcja A i ortogonalność Q . Skala log na osi y.

8 Wnioski

- Złożenie rotacji Givensa daje poprawny rozkład QR w sensie numerycznym: QR odtwarza losową macierz testową z błędem na poziomie maszynowym, a Q pozostaje prawie ortogonalna ($\|Q^T Q - I\|_F$ jest mikroskopijny).
- Wyniki funkcji `qr` są porównywalne co do $\|A - QR\|_F$; różnice w samych macierzach Q i R mogą wynikać z niejednoznaczności znaków, nie z błędów metody.
- Pomiary czasu rosną z n zgodnie z intuicją kosztu sześciennego; pojedyncze fluktuacje to raczej hałas pomiaru niż właściwość samego wzoru na c, s .
- Wykres błędów vs. n na skali logarytmicznej utwierdza, że procedura nie „psuje się” gwałtownie przy powiększaniu rozmiaru w rozpatrywanym zakresie — błąd utrzymuje się około epsilon maszynowego w normie Frobeniusa.